

Common Problems

Video Recommendations 🎬

By Stefan Mai · Published May 1, 2025 · 🏷️ hard

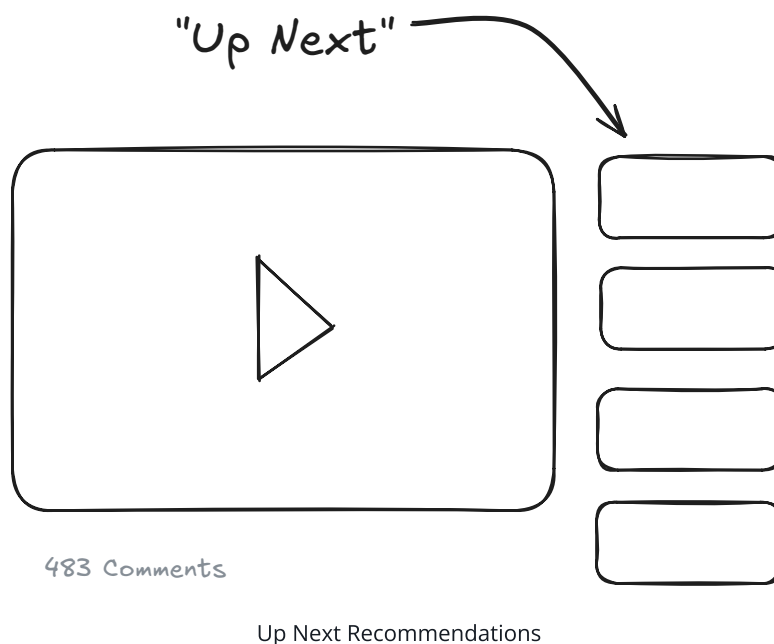
Understanding the Problem

🔍 What is YouTube?

YouTube is a video sharing platform where creators upload videos and users can watch, like, comment, and share them. Users discover videos through subscriptions, search, recommendations on their homepage, and "up next" suggestions while watching videos. The platform serves billions of video views daily across diverse content categories.

Video recommendation systems are at the heart of modern video platforms. They help users discover content they'll enjoy from an enormous catalog of videos, while helping creators reach their audience. It's no understatement to say that large video platforms cannot succeed today without them.

For this problem, we'll focus on YouTube's video recommendation system, specifically the "up next" recommendations. Recommendation systems are a very popular interview topic and this is a great example of a system that has a lot of moving parts.



Problem Framing

Let's start by establishing a clear framing for the problem we're trying to solve.

Clarify the Problem

We'll begin by asking our interviewer key questions to understand the scope and constraints. Some of these constraints don't even require questions and more senior engineers will be able to make assumptions (and confirm them with the interviewer). Your goal as a candidate here is to sufficiently understand the problem at hand to be able to make a design. If it still seems unclear, keep asking questions until you're confident you understand the problem.

- What types of recommendations do we need to generate?
 - *Let's focus on the "up next" recommendations shown while watching a video.*
- How many videos are there? Is it safe to assume around 1B?*
- *1B sounds like a fine estimate for videos.*
- How many users do we have?
 - *Let's assume 1B daily active users.*
- How many videos are shown to the user on the screen at once, 5, 10?
 - *Let's work with 5 for now.*
- How quickly do we need to show recommendations when the user views a video?
 - *We display within a small window, say 250ms.*

Let's capture these key points on our whiteboard:

1B+ videos
1B DAU, slate of 5 recommended videos
250ms to serve
Recommendations are in context of an existing video

Problem Clarification

Establish a Business Objective

Next, we have to come up with an overall business objective. This is distinct from our ML metrics and represents what success looks like for the business. Recommendation systems are uniquely positioned to impact the business, but they can succumb to optimization pitfalls without a clear objective, so the discussion here is both important and signals experience to your interviewer.

Bad Solution: Maximize click-through rate (CTR)



Good Solution: Maximize total watch time



Good Solution: Maximize quality-adjusted watch time



Great Solution: Maximize long-term satisfaction and engagement



Most interviewers will be fine with any "good" objective here, but you can stand out by showing longer-term thinking. Having a business objective that is more "pure" gives more room for creativity and depth — which becomes more important for senior+ candidates where the "inner" loop is more obvious and mature.

We'll proceed with maximizing quality-adjusted watch time as our business objective.



Notice how this objective helps answer prompt some deep dives and challenges. For example, "how do we avoid recommending a clickbait video that gets lots of initial clicks but poor retention?" becomes clearer when we consider long-term engagement. Similarly, it opens up some interesting discussion around "outer loop", long-range optimizations like creator promotion and platform health.

Decide on an ML Objective

With our business objective defined, we can now specify our ML objective. Our *primary* objective is a ranking problem: given a user and context, we need to rank available videos by their likelihood of contributing to long-term satisfaction and engagement. The context for the

ranking problem is the user's current session and (very importantly) the video they are watching. We expect our system to be responsive as the user demonstrates their intent by browsing through videos.

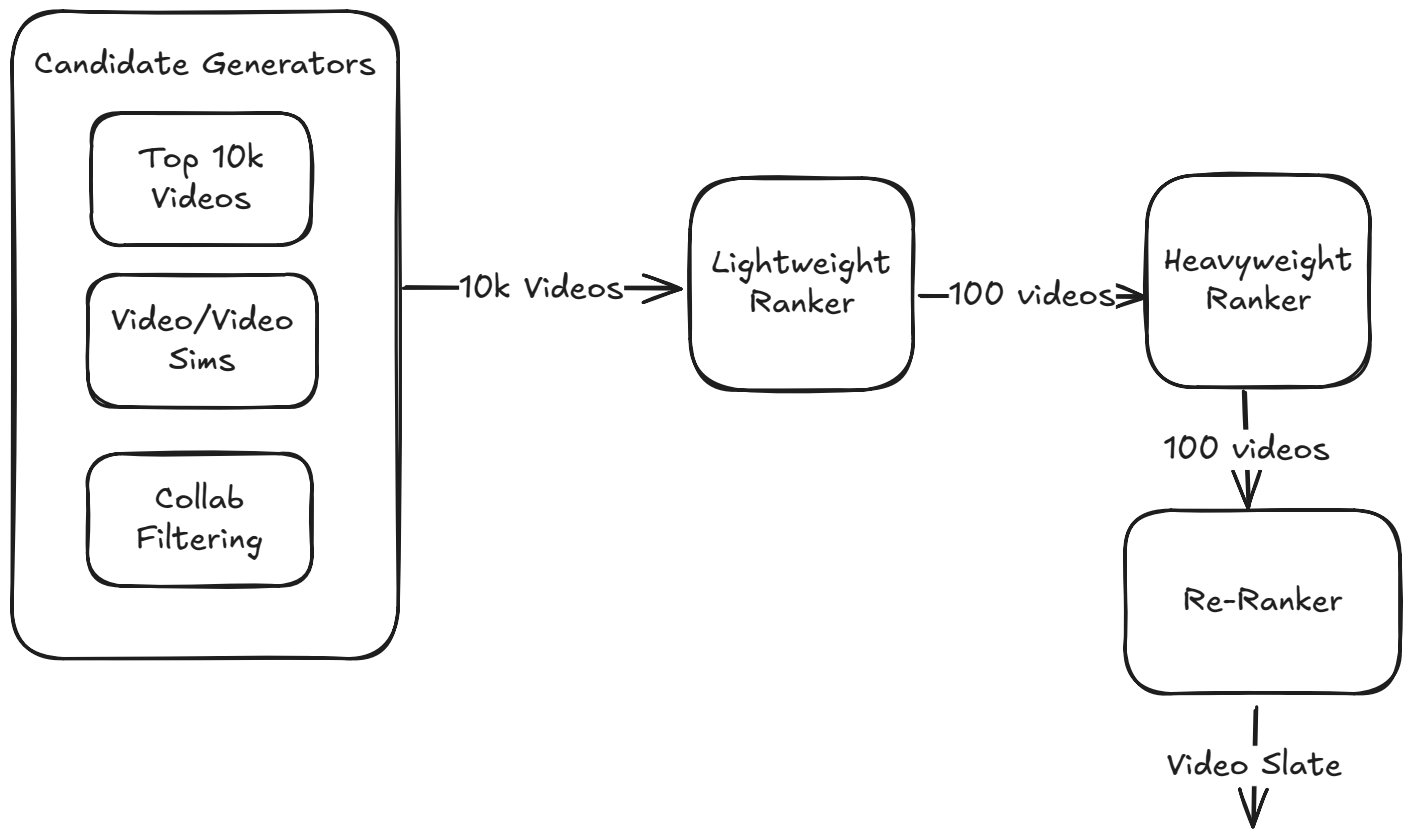
Our ranking is mostly a function of our predictions about future behavior. Are they going to click? Will they spend time watching the video? Will they share it? How we trade off these predictions will be at the heart of our "value model" which helps us to tune our system at a high level toward our business objective.

But for this problem ranking is only part of the story. To meet our scaling requirements, we're going to need an architecture that allows us to rank billions of videos for billions of users. That sounds hard and important, so to make this clear to our interviewer we'll take to sketching out what the overall system might look like before we dive into the details of interesting and important pieces.

High Level Design

Modern recommendation systems at scale almost universally use a multi-stage architecture and it's become relatively standard in industry. If you've never worked on a rec system before, this is likely to be news to you which can be a bit unfair in an interview. Fortunately, you're learning about it here, but expect that some aspects of this will be "table stakes" and taken for granted from your interviewer.

To start, we're going to draw some big boxes to represent the different stages and talk about how they work together in our specific video recommendation system. The goal here isn't to complete our design but to show the interviewer how the pieces fit together before we dive into the details.



High Level Design

We'd quickly talk through each component to be clear:

First, we have our candidate generation layer. This consists of multiple generators running in parallel to produce candidate videos that we'll rank, in practice we may have several hundred generators. Candidate generators have variable context: some might be universal (e.g. "top 10k platform videos") while others might be personalized (e.g. "videos from the user's subscriptions").

We can reserve the rest of the discussion about candidate generators for the features section as there's significant correspondence between candidate generators and the features we'll use in our ranking model. In short: if it's an informative feature, we probably need to ensure we have candidates covering it.



Candidate generators with limited context are highly cacheable and can be pre-computed. We'll want to note this to our interviewer as it's a key ingredient for scaling.

The outputs of these generators feed into our lightweight ranker. This model serves primarily as a computational optimization, using fast-to-compute features to reduce our candidate set

from $O(10k)$ videos to $O(100)$. The key here is to optimize for recall. We want to make sure we don't discard any videos that might end up being great recommendations.

Next comes our heavy ranking model, which is where the real magic happens. This model will use our full set of features to score the videos, learning useful representations of user preferences and videos together with their interactions. We'll discuss those in a bit.

Finally, we have our re-ranking layer which optimizes for the overall recommendation slate. This is where we apply our value model to balance user engagement, creator success, and platform health, ensure diversity, and handle special cases like new creator promotion or viral content.



While this architecture is powerful, it's not perfect. Each stage can introduce biases, and the system can still miss good recommendations if the candidate generators aren't properly tuned. Monitoring and maintaining each stage is crucial for system health and it's very common for interviewers to ask about this.

Data and Features

Now that we have a scaffolding for the overall design, let's discuss the data and features we'll use to train our models. This is particularly rich for video recommendations as we have multiple types of data available.

Training Data

Recommendation systems exist in a glut of behavioral data which can be used as supervision for models. This can make this section of the interview tricky because it's easy to get lost in unimportant discussions about the data. What we want to communicate to our interviewer is that we understand the general landscape of data available to us and that we can thoughtfully create hypotheses about what data is going to be most useful for our system.

To do this, we're going to break our training data into categories and talk about a few representative examples in each category. This saves us from having to be exhaustive but demonstrates to our interviewer that we generally understand what's useful. We want to **keep moving**.

Explicit Feedback

Our most valuable supervision come from explicit user actions:

- Likes and dislikes
- Subscriptions
- "Not interested" feedback

These are high-quality signals but relatively rare. The majority of users consume content passively without providing explicit feedback.



Some candidates will assume things like explicit user "interests" would be useful, but generally speaking this isn't true. Users often don't have a solid handle on what they truly enjoy and the gap between their explicit preferences and "revealed" preferences can be quite large. We should prefer those signals that don't require the user to think too much!

Implicit Feedback

The majority of our training data will come from implicit feedback. This is feedback that we can infer from user behavior but they aren't making explicit statements about their preferences.

- Watch time (both absolute and relative to video length)
- Click history
- Sharing behavior
- Whether users return to the platform after watching



Videos which cause user attrition (e.g. they leave the platform) are a goldmine for training data as they are some of the strongest implicit negative signals we can get. Users who go to another video might dislike it, but users who "nope I'm done" are telling us something stronger (with noise).

Contextual Data

Beyond direct feedback, we have rich contextual information:

- Details about the current video they are watching (e.g. creator, title, semantics, etc.)
- Previous search queries
- Time of day and day of week
- Device type and form factor
- Previous videos watched in the session

This data helps us understand the context in which recommendations are consumed and can significantly impact their relevance.

Features

From our training data, we can derive features that will be useful for our models. We'll organize these into logical groups based on their source and update frequency, with the goal of maximizing cacheability and minimizing latency when we need to re-generate recommendations. Each of our rankers will take as input two legs: the context of the recommendation (e.g. the video the user is currently watching, the user's profile and history, etc.) and each of the candidate videos we've generated.



Feature discussions can get protracted in ML system design interviews, and there's a lot of other details to get into in the interview, so we'll focus on a few indicative features but keep things brief. We can indicate our bias for the interviewer and ask if they want additional detail "there's a lot I want to cover so I'll stop here; let me know if you want me to go into further detail on features, there's clearly a lot more we can discuss".

Video Features

Our video features can apply to both the context video and the candidate videos. Our assumption is that the user is currently watching a video they were interested in, so it probably gives us some insight into what they want next.

Content-based features can be computed once, when the video is uploaded or edited, and cached for a long time.

- Video metadata (title, description, tags)
- Thumbnail features (extracted via computer vision)

- Audio features (music, speech, etc.)
- Video quality metrics (resolution, stability)
- Topic and category embeddings



It can be hard to know whether people will enjoy a video based on its content alone. You might be able to summarize broadly what the topic is or whether the production quality is high, but generally speaking recommendation systems favor responding to user behavior vs cold readings of the videos themselves.

Engagement features are more dynamic and will need to be updated more frequently.

- Historical engagement metrics (views, likes, average watch time)
- Velocity metrics (recent growth in views/engagement)
- Creator reputation and historical performance
- Monetization status and advertiser friendliness



Engagement metrics need careful normalization. A video that's 30 seconds old will naturally have fewer views than one that's been live for a day. Using velocity metrics and comparing to expected growth curves can help normalize these signals.

It's common to have some discussion here about feedback loops, we want to avoid runaway recommendations that are just popular because they're popular.

User Features

Next, we'll want features we can use to represent the user and their preferences.

First, we have some **profile features** that are static and don't change much. These might be explicitly provided by the user or inferred (perhaps using additional models) from their behavior and we'll update them as the inputs change:

- Topic preferences
- Language preferences
- Subscription list embeddings

- Demographics (if available)

Next we have **behavioral features** that represent the user's recent behavior. This is the most dynamic data and will need to be updated frequently:

- Session-based features (recently watched videos, searches)
- Long-term preferences (favorite creators, categories)
- Time-of-day and day-of-week patterns
- Device and platform preferences

Depending upon our choice of model, we're going to need to do some work to encode/represent these features. This is a common probe from interviewers and often worth a bit of proactive discussion to avoid miscommunication. As an example, for recently watched videos we might summarize this in our lightweight ranker by using an average of the embeddings of the videos, potentially with different time windows (last 10, last 3). For our heavy ranker we'll want to be able to make sense of the full sequence.

Modeling

Now that we have our data and features defined, let's discuss our modeling approach.

Benchmark Models

If we're launching this system for the first time, we'll want to start with some simple models that can serve as baselines. Some easy baselines might be to use a random blending of our candidate generator outputs or a simple collaborative filtering approach which picks videos given a user. We expect these to underperform and if they don't we'll have a really good starting position to ablate from. It also gives us a good data point for the tradeoff we'll be navigating between compute and recommendation quality.

While you likely won't elaborate on these solutions in great detail, it's good to be able to mention how you'd establish benchmarks as a good practice which demonstrates practical experience.

Model Selection

With our baselines in place, we can now proceed with our more sophisticated approach. For this problem, we'll need separate models for candidate generation and ranking.

Many of our candidate generators will be thin interfaces on vector databases. We'll query the database with an embedding (usually the current video or the current user, but could also be the last video from the current user, etc.) and retrieve the top K closest vectors in the database. These items are then passed on to our ranking stages. We'll talk about this process in a moment.

For our ranking models, we have different priorities:

- Light ranker: high recall, low latency, high throughput
- Heavy ranker: high precision, high quality, lower throughput

Our light ranker is often a tree-based model like a GBDT (gradient boosted decision tree), a very skinny MLP (multi-layer perceptron), or a combination of the two. XGBoost and LightGBM routinely top the leaderboards in recommendation system benchmarks. These models can run on CPU (economical to scale) with sub-millisecond latencies that allow them to churn through the billions of candidates they'll see in our candidate generation stage. An MLP-based approach can be valuable because we can distill the model from our heavy ranker and take advantage of wide embeddings we've learned from categorical features. For our interview, we'll pretend GPUs are in short order (aren't they always?) and use a tree-based model.



This is a good example of a place where "classical" ML approaches can still be competitive. Environments with heavy compute pressure and lower performance requirements can be a great place to flex if you have knowledge of these approaches.

For our heavy ranker, we have some options:

Bad Solution: Simple MLP



Good Solution: Deep Learning Recommendation Model (DLRM)



Great Solution: Transformer-based Sequence Ranker



There's a very good chance that someone unfamiliar with recommendation systems won't have heard of DLRM, but interviewers will generally expect senior candidates to be observant of the transformer revolution and to have a bias toward a transformer-based approach. For simpler MLP-style approaches, acknowledging the heterogeneity of sparse/dense features is an important way to demonstrate deeper modeling experience.



A lot of material online about recommendation systems will focus on matrix factorization or collaborative filtering models. These aren't state of the art anymore and most applications of importance have graduated to more sophisticated approaches.

Model Architecture

Let's detail the architecture for each stage:

Candidate Generation Models

For some of our embedding-based candidate generation models, we'll train them using a two-tower architecture with a triplet loss. This is a common approach for retrieval systems and is well-suited to our use case.

If we're building a "videos this user will like" generator, we'll assemble a dataset of triplets of the form `(user, positive_video, negative_video)`. Two parallel towers will be trained to produce embeddings for the user and the candidate videos. The loss function will then be:

$$L = \max(0, \text{margin} + d(\text{user}, \text{positive_video}) - d(\text{user}, \text{negative_video}))$$



where `d` is the distance function (often the dot product of the embeddings). During training we'll sample negative videos and bias toward those negatives which are "hard" (videos we expect the user to like, but they don't).

The embeddings for both the user and the candidate videos will then be stored in our vector database for retrieval.



Whether you'll need to explain much of this is a function of how much your interviewer believes you to know the details. Interviewers are trained to sniff out a lack of confidence and try to push until they've found the bottom of your knowledge. It's not uncommon for candidates to hand-wave about embeddings without having clear knowledge of how they're trained.

Heavy Ranking Model

For our final ranking, we'll use a transformer-based architecture optimized for multiple prediction tasks. The key insight here is that different aspects of user engagement (watch time, likes, shares, etc.) are all correlated but provide different signals about content quality and user satisfaction. By training our model to predict multiple outcomes simultaneously, we can learn richer representations that capture different aspects of user behavior.

The architecture consists of a lot of pieces which you'll probably discuss sparsely with your interviewer.

1. Input Processing:

- Embedding layers for categorical features (video_id, creator_id, etc.)
- Normalization layers for numerical features (view counts, engagement rates)
- Sequence processing for historical features with positional encoding
- Special tokens to denote different types of user actions (watch, like, share)

2. Feature Interaction:

- Cross-attention layers to model interactions between user history and candidate videos
- Self-attention layers to capture patterns within user history
- Feed-forward networks to process the attended information
- Residual connections and layer normalization for stable training

3. Output Layers:

Multiple classification and regression heads, each specialized for different prediction tasks:

- Watch time prediction (regression)
- Click-through probability (binary classification)

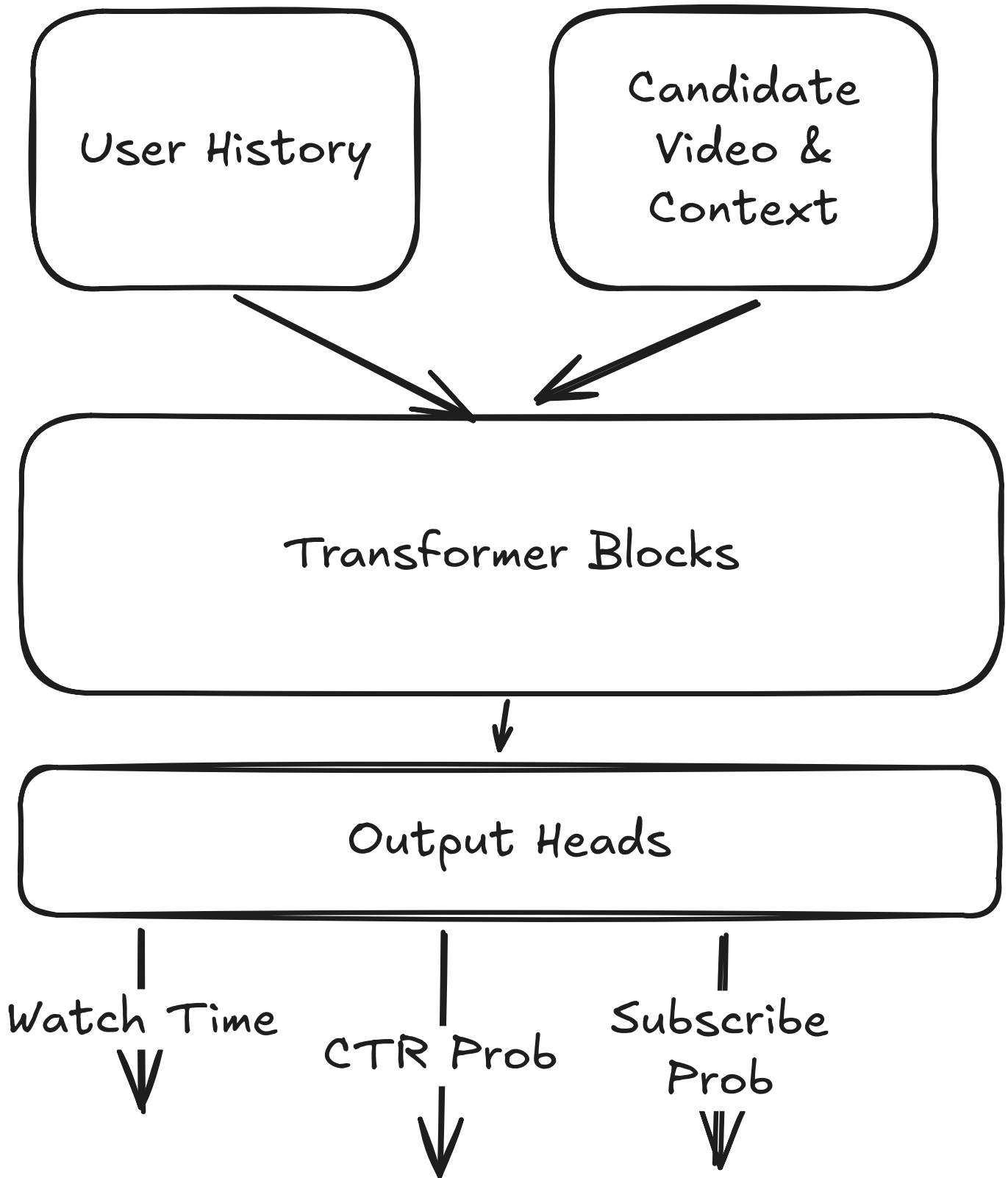
- Like probability (binary classification)
- Share probability (binary classification)
- Completion rate (regression)
- Return visit probability (binary classification)

The multitask setup serves several purposes:

- Acts as a regularizer, preventing overfitting to any single metric
- Provides auxiliary signals during training
- Each of the outputs can be potential inputs to our value model in re-ranking
- Enables better cold-start handling by leveraging correlations between tasks



The choice of prediction heads is a potential opportunity to show off. For example, if creator growth is important, you might add heads for predicting subscriber conversion or creator retention metrics.



Model Architecture

Loss Function

Our loss function needs to balance multiple objectives:

1. Primary Engagement Loss:

$$L_{\text{engage}} = -\sum(y_{\text{true}} * \log(y_{\text{pred}}) * \text{watch_time_weight})$$

where `watch_time_weight` is a function of both absolute and relative watch time

2. Auxiliary Tasks:

$$L_{\text{aux}} = \alpha * L_{\text{click}} + \beta * L_{\text{completion}} + \gamma * L_{\text{satisfaction}}$$

These help the model learn better representations

3. Position Bias Correction:

$$L_{\text{position}} = \delta * \text{BCE}(y_{\text{true}}, y_{\text{pred}}) * \text{position_weight}$$

Corrects for position bias in historical data

The final loss is a weighted combination:

$$L = L_{\text{engage}} + L_{\text{aux}} + L_{\text{position}}$$



It's a common interview question to have follow-up questions about how to handle presentation bias. Remember that the training examples we have are generated from our users, but also the current system. Without debiasing, newly trained models will eagerly try to approximate *the output of the current system* instead of learning the true user preferences. Position bias correction is one technique to address this.

Inference and Evaluation

Inference System

Our inference system needs to handle massive scale while maintaining low latency. The depth of this section will depend in part of the position you're interviewing for. This is where the line between applied ML and ML infra becomes a bit blurry.

Our system has a good architecture for scaling, we've separated ranking from candidate generation to make it tractable and inserted a lightweight ranker to bring it closer to optimal.

A lot of processing can be done offline. For example, we can pre-compute the embeddings for all of our videos and cache them. We can also cache the embeddings for our users and cache the results of our candidate generation. Modern systems (like [Bytedance's Monolith](#)) offer some interesting techniques for enabling online, efficient updates.

For serving our models, we can use techniques like quantization to reduce the memory footprint of our models. Leveraging GPU/TPU hardware will also allow us to serve models with higher throughput.

Evaluation Framework

Our evaluation strategy needs to consider both offline and online metrics. Offline metrics are useful, but fraught, since we're trying to approximate user behavior which is inherently a moving target. We want metrics that give us a useful signal about whether or not an online experiment might be successful.

Our ranking is inherently a function of predicting various engagements (these are our prediction heads). Each of these heads can be evaluated separately.

The final ranking outputs can be evaluated by looking at standard ranking metrics like NDCG, MAP, and precision/recall. We should also look at the diversity of the recommendations, both in terms of the videos and the creators.

Each stage in our system is a new source of potential error in later stages, so understanding (e.g.) the recall of our candidates generators is also an important evaluation. To do this effectively, we need unbiased inputs — ideally we're ranking candidates that are either outside the window (e.g. beyond the 10k candidates we surface) or outside the scope of a given candidate generator altogether.

Online Evaluation

Our gold standard for online evaluation is A/B testing. We can test our models against the current system and see how they perform. We can also test our models against each other to see which ones are better.

We can track a variety of metrics here:

1. A/B Testing Metrics:

- Session watch time
- Return visit rate
- Long-term engagement trends
- Creator satisfaction metrics

2. User Experience Metrics:

- Recommendation acceptance rate
- Survey feedback
- Negative feedback rate

3. System Health Metrics:

- Latency
- Throughput
- Resource utilization
- Error rate



Experiments frequently suffer from "novelty effects" where the new system is novel and the results are better than they would be otherwise. If the old system was serving you A videos (which you're tired of) and the new system serves you B videos, you might see a bump in engagement that lasts until you also tire of B . Designing these experiments carefully is important.

Deep Dives

There is so much to talk about in this interview that typical candidates will only cover a small number of deep dives (either proactively driven or prompted by the interviewer). It can be helpful to have a shortlist of topics you've accumulated over the session to propose. This gives

your interviewer signal "ah, they noticed this and didn't forget it" without you needing to necessarily cover the details of it.

"I see we've got only a few minutes left, I wanted to talk about cold start, explore/exploit, and feedback loops. Do you have any preference on which one we start with?"

Feedback Loops

Recommender systems create feedback loops that impact user experience. The primary concerns include popularity bias (the "rich-get-richer" effect where highly-ranked videos receive more exposure), filter bubbles (users seeing increasingly narrow content selections), and creator behavior optimization (content producers focusing on algorithm-friendly formulas rather than quality).

To mitigate these issues, we can implement counterfactual logging (logging the user's feedback on a video that was not shown) and inverse-propensity reweighting to debias training data, design exposure-penalized loss functions to prevent overexposure, and enforce diversity constraints at the slate level. Periodic offline refreshes with uniformly-sampled impressions help prevent popularity bias.

In production, monitoring diversity metrics and creator success distribution is essential to detect unhealthy patterns early. By balancing engagement with diversity and novelty in a multi-objective framework, we can maintain a recommendation system that serves both users and content creators effectively over time.

Cold Starting Users/Videos

When new users join our platform or fresh videos are uploaded, we lack the behavioral data that powers our heavy ranker. This is only going to impact a sliver of users and videos, but they're the most vulnerable ones and pivotal to platform growth. Lots to discuss.

To address this challenge for new users, we can leverage demographic information and initial onboarding preferences to place them into coarse clusters with similar existing users. It's not uncommon to have a special flow where users select channels to subscribe to, or answer questions about their interests. This allows us to bootstrap recommendations based on what has worked well for similar users until we gather enough interaction data to personalize further.

For new videos, we can extract rich features from the content itself using multimodal models that analyze thumbnails, titles, descriptions, transcripts, and even the video content. Additionally, we can implement a controlled exploration strategy where we expose new videos to a diverse but limited audience to quickly gather initial engagement signals without risking poor recommendations to our broader user base. The key here will be to ensure that, once we have some behavioral signal, we can shift to a more personalized approach.

In an interview, you might go into one or two approaches in more depth.

Explore/Exploit Tradeoffs

We need to balance showing users stuff we know they'll like (that's the exploitation part) versus throwing in some wild cards that might uncover new interests (that's exploration). Get this balance wrong, and you're either stuck in a boring content loop or annoying users with random stuff they hate.

In the real world, this is tackled with some clever techniques like Thompson sampling or contextual bandits. A good approach is setting up a multi-armed bandit system where each recommendation spot gets its own risk budget. This way, we can play it safe with the top recommendations but get a little crazy with the ones further down the page. We can also be more adventurous with new users and dial it back as we learn more about them. For videos specifically, if someone's all over the place with cooking videos but super consistent with cat videos, we might explore more in cooking and stick to the cat content they clearly love.

In an interview setting, senior candidates might be asked to discuss the mathematical foundations of these approaches, such as how Upper Confidence Bound (UCB) algorithms work or how to implement epsilon-greedy strategies at scale. Interviewers often probe into how you would measure the effectiveness of your exploration strategy, what metrics you'd track to ensure you're not sacrificing too much short-term engagement for long-term discovery, and how you'd adapt your approach based on user segments. They might also ask about the operational challenges of implementing these algorithms in a production environment, including how to handle the computational overhead of maintaining confidence intervals for millions of items or how to ensure exploration doesn't disproportionately affect certain user groups.

What is Expected at Each Level?

Ok, that was a lot! Let's take a step back and talk a bit about what interviewers tend to expect.

For this problem, mid-level engineers are expected to demonstrate practical proficiency with recommendation systems and their core components. They should be able to articulate the basic architecture of a recommendation system, including candidate generation and ranking stages. Mid-level engineers should show familiarity with common engagement signals (likes, watch time, CTR) and how to incorporate them as features. They need to understand standard evaluation metrics like NDCG and MAP, and demonstrate awareness of the scaling challenges inherent in serving recommendations to billions of users. Generally speaking, mid-level engineers will differentiate themselves by showing they can implement a working recommendation system using established patterns and best practices, even if the system isn't state of the art.

Senior-level engineers will need to demonstrate significantly more depth in their understanding of recommendation systems. Their expertise in feature engineering should be apparent in how they handle data quality issues, normalize engagement signals, and address temporal aspects of the data. They should have experience with multi-stage architectures and be able to articulate the tradeoffs between different approaches. Senior engineers should demonstrate strong knowledge of serving optimizations like caching strategies, embedding compression, and efficient nearest neighbor search. Most importantly, they should show they can balance competing objectives - understanding how to trade off metrics like engagement, diversity, and creator success. They'll often bring up practical considerations around A/B testing, monitoring, and maintaining recommendation quality over time.

Staff-level candidates are expected to demonstrate mastery of recommendation systems at both technical and strategic levels. They should quickly establish the core architecture and then focus on the most impactful and challenging aspects of the system. Staff engineers will often identify and propose solutions to systemic issues like feedback loops (popular content getting more recommendations and thus more engagement), cold-start problems (how to recommend new content or serve new users), and data quality challenges (dealing with position bias, missing data, and noise in user feedback). They'll have a deep understanding of how recommendation biases can affect both user experience and creator ecosystems, and will propose creative solutions to measure and mitigate these effects. Staff-level candidates usually recognize that the bottleneck in modern recommendation systems isn't just model architecture and that it's often about data quality, evaluation methodology, and overall system design.

References

- [Improving Recommendation Systems & Search in the Age of LLMs](#)
- [Graph Neural Networks in Recommender Systems: A Survey](#)
- [Transformers for Recommender Systems in Fashion Ecommerce](#)
- [Deep Learning Recommendation Model for Personalization and Recommendation Systems](#)
- [Monolith: Real Time Recommendation System with Collisionless Embedding Table](#)